

10-09-2023

Design and Analysis of  
Algorithms

MUHAMMAD HASSAN KHALID

21L-5692

BOS-5A

Q1  $\Rightarrow 4n^2 < 32n \lg n$   
 $\Rightarrow \frac{4n^2}{4} < \frac{32n \lg n}{4}$

$\Rightarrow n^2 < 8n \lg n$

$\Rightarrow \frac{n^2}{n} < \frac{8n \lg n}{n}$

$\Rightarrow n < 8 \lg n$

$\Rightarrow \frac{n}{8} < \frac{8 \lg n}{8}$

$\Rightarrow \frac{n}{8} < \lg n$

$\Rightarrow 2^{n/8} < n$

For  $n=8 \Rightarrow 2^{8/8} < 8$   
 $2 < 8$  (TRUE)

For  $n=16 \Rightarrow 2^{16/8} < 16$   
 $4 < 16$  (TRUE)

For  $n=32 \Rightarrow 2^{32/8} < 32$   
 $16 < 32$  (TRUE)

For  $n=64 \Rightarrow 2^{64/8} < 64$   
 $256 < 64$  (FALSE)





$$\text{For } n=48 \Rightarrow 2^{48/8} < 48 \\ 64 < 48 \text{ (FALSE)}$$

$$\text{For } n=40 \Rightarrow 2^{40/8} < 40 \\ 32 < 40 \text{ (TRUE)}$$

$$\text{For } n=44 \Rightarrow 2^{44/8} < 44 \\ 44.8 < 44 \text{ (FALSE)}$$

$$\text{For } n=42 \Rightarrow 2^{42/8} < 42 \\ 38.4 < 42 \text{ (TRUE)}$$

$$\text{For } n=43 \Rightarrow 2^{43/8} < 43 \\ 42.4 < 43 \text{ (TRUE)}$$

So, at  $n=44$ , merge sort starts to beat insertion sort again. Therefore, for  $2 \leq n \leq 43$ , insertion sort beats merge sort.

Q2

$$100n^2 < 2^n$$

$$\text{For } n=1 \Rightarrow 100(1)^2 < 2^1 \\ 100 < 2 \text{ (FALSE)}$$

$$\text{For } n=2 \Rightarrow 100(2)^2 < 2^2 \\ 400 < 4 \text{ (FALSE)}$$

$$\text{For } n=3 \Rightarrow 100(3)^2 < 2^3 \\ 900 < 8 \text{ (FALSE)}$$

$$\text{For } n=5 \Rightarrow 100(5)^2 < 2^5 \\ 2500 < 32 \text{ (FALSE)}$$



$$\text{For } n=7 \Rightarrow 100(7)^2 < 2^7$$

$$4900 < 128 \text{ (FALSE)}$$

$$\text{For } n=13 \Rightarrow 100(13)^2 < 2^{13}$$

$$16900 < 8192 \text{ (FALSE)}$$

$$\text{For } n=14 \Rightarrow 100(14)^2 < 2^{14}$$

$$19600 < 16384 \text{ (FALSE)}$$

$$\text{For } n=15 \Rightarrow 100(15)^2 < 2^{15}$$

$$22500 < 32768 \text{ (TRUE)}$$

So, at  $n=15$ , A starts to run faster than B.

Q3 a

```
int s, i, n; —  $O(1)$ 
cin >> n; —  $O(1)$ 
s = 0; —  $O(1)$ 
for (i = n; i >= 1; i--) —  $O(n)$ 
    s++; —  $O(1)$ 
```

$$T(n) = O(1) + O(1) + O(1) + \overset{O(n)}{\cancel{O(1)}} + O(1)$$

$$= O(n)$$

$$\text{Let } c = 1$$

$$T(n) \leq 1 * n \text{ (TRUE)}$$

b

```
int sum, i, j, n; —  $O(1)$   
sum = 0; —  $O(1)$   
cin >> n; —  $O(1)$   
for (i = 1; i < n; i = i * 2) —  $O(\log(n))$   
    for (j = 1; j < n; j = j * 2) —  $O(\log(n))$   
        sum++; —  $O(1)$ 
```

$$T(n) = O(1) + O(1) + O(1) + O(\log(n)) + O(\log(n)) + O(1)$$

$$T(n) = O(\log(n))$$

$$\text{If } c=1; T(n) \leq 1 * \log(n) \text{ (TRUE)}$$

c

```
int n = 0, j = n; —  $O(1)$   
while (j > 0) { —  $O(\log(n))$   
    n += j * 3; —  $O(1)$   
    j /= 4; —  $O(\log_4 n)$   
}
```

$$T(n) = O(1) + O(\log(n)) + O(1) + O(\log_4(n))$$

$$T(n) = O(\log(n))$$

$$\text{If } c=1; T(n) \leq 1 * \log(n) \text{ (TRUE)}$$



Q4

Pseudocode:-

Selection Sort (A):

for i from 0 to n-2 do

    min\_index = i

    for j from i+1 to n-1 do

        if  $A[j] < A[\text{min\_index}]$  then

            min\_index = j

    swap  $A[i]$  and  $A[\text{min\_index}]$

The algorithm only needs to run for the first n-1 elements instead of all n elements is that when the n-1 smallest elements are in their sorted positions, the nth (largest) element is already in its correct position by default, as there is no other place left for it in the array. Therefore, there's no need to explicitly sort it.

Running Time Analysis!

Best-Case Running Time:-  $\Theta(n^2)$

The best case occurs when the input array is already sorted, and the algorithm still performs n-1 iterations of the outer loop and checks the inner loop each time.

Worst-Case Running Time:-  $\Theta(n^2)$

The worst case occurs when the input array is sorted in reverse order, as the

algorithm will perform  $n-1$  iterations of the outer loop, and for each iteration, it will compare with and swap elements in the inner loop.

The selection sort algorithm has a quadratic time complexity in both the best and worst cases, which is  $\Theta(n^2)$ .

Q5

$$c_1 \cdot n^3 \leq T(n) \leq c_2 \cdot n^3 \text{ for all } n \geq n_0$$
$$c_1 \cdot n^3 \leq \frac{1}{8} n^3 - 5n^2 \leq c_2 \cdot n^3$$

$$\Rightarrow c_1 n^3 \leq \frac{1}{8} n^3 - 5n^2$$

$$\Rightarrow \frac{c_1 n^3}{n^3} \leq \frac{\frac{1}{8} n^3}{n^3} - \frac{5n^2}{n^3}$$

$$\Rightarrow c_1 \leq \frac{1}{8} - \frac{5}{n}$$

$$\Rightarrow c_1 \leq \frac{1}{8} \text{ (as } n \text{ approaches infinity)}$$

$$\Rightarrow c_1 = 0$$

$$\Rightarrow \frac{1}{8} n^3 - 5n^2 \leq c_2 \cdot n^3$$

$$\Rightarrow \frac{\frac{1}{8} n^3}{n^3} - \frac{5n^2}{n^3} \leq \frac{c_2 n^3}{n^3}$$

$$\Rightarrow \frac{1}{8} - \frac{5}{n} \leq c_2$$

$$\Rightarrow c_2 \geq \frac{1}{8} \text{ (as } n \text{ approaches infinity)}$$



As there exists constants  $c_1 = 0$

and  $c_2 = \frac{1}{8}$  such that :

$$0 \leq T(n) \leq \frac{1}{8} n^3 \text{ for all } n$$

sufficiently large, we can conclude that  $T(n)$  is indeed  $\Theta(n^3)$ .

Q6

Function Mystery (n)

{

if (n > 1)

{

Print "hello"

Mystery (n/5) —  $O(n/5)$

For (i = 1 ... n) —  $O(n)$

Print "world"

Mystery (2n/5) —  $O(2n/5)$

}

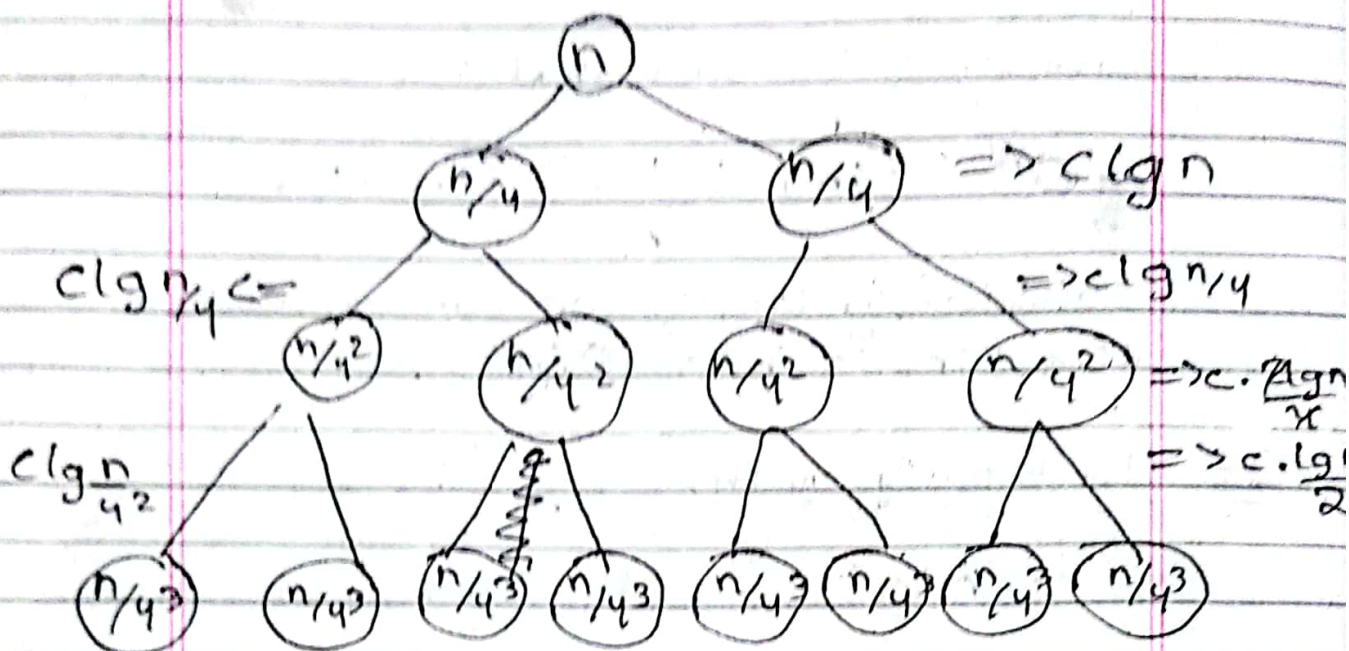
}

$$T(n) = O(n/5) + O(n) + O(2n/5)$$

$$T(n) = O(n)$$

Therefore, the runtime of the function Mystery (n) is  $O(n)$ .

Q7  $T(n) = 2T(n/4) + O(\lg n)$



$$\Rightarrow c \lg \frac{n}{4^2} + c \lg \frac{n}{4^2} + c \lg \frac{n}{4^2} + c \lg \frac{n}{4^2}$$

$$\Rightarrow c \cdot 4 \frac{\lg n}{4^2} \Rightarrow c \cdot \frac{\lg n}{4}$$

$$c \lg n \left[ 1 + \frac{1}{2} + \left(\frac{1}{2}\right)^2 + \dots \right]$$

$$c \lg n \left[ \frac{1}{1 - 1/2} \right]$$

$$c \frac{12}{11} \lg n$$

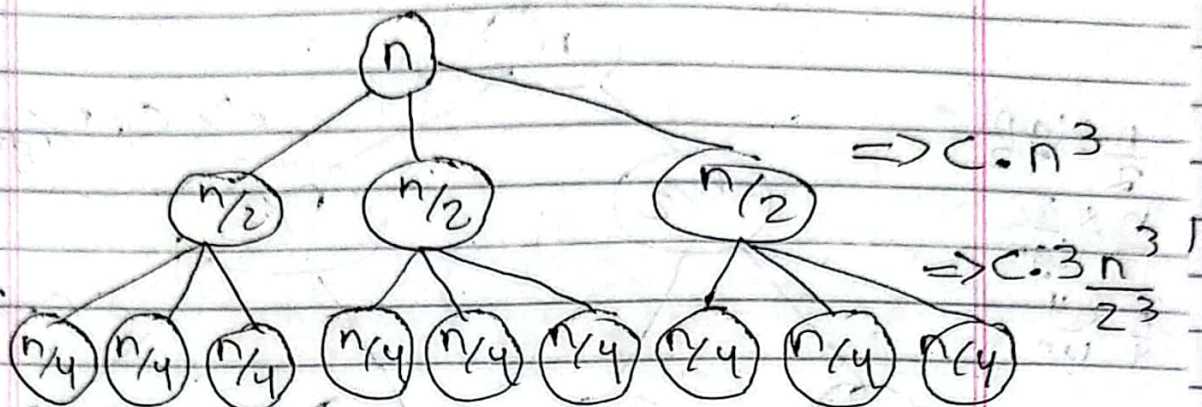
$$= O(\lg n)$$

The asymptotic upper bound is  $O(\lg n)$ .



b

$$T(n) = 3T(n/2) + O(n)^3$$



$$\Rightarrow C \cdot n^3 \left[ 1 + \frac{3}{2^3} + \frac{3^2}{2^6} + \frac{3^3}{2^9} \dots \right]$$

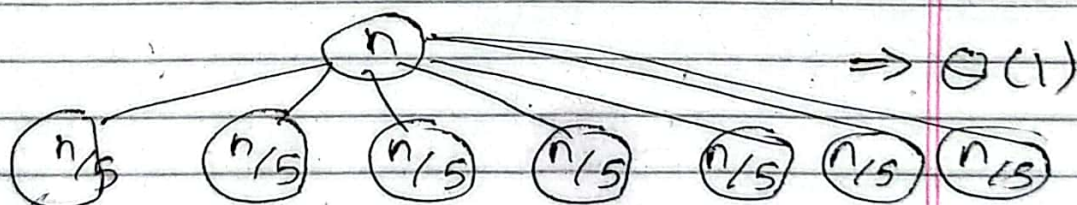
$$\Rightarrow C \cdot n^3 \left[ \frac{1}{1 - (3/2)} \right]$$

$$\Rightarrow O(n^3)$$

The asymptotic upper bound is  $O(n^3)$ .

c

$$T(n) = 7T(n/5) + \Theta(1)$$



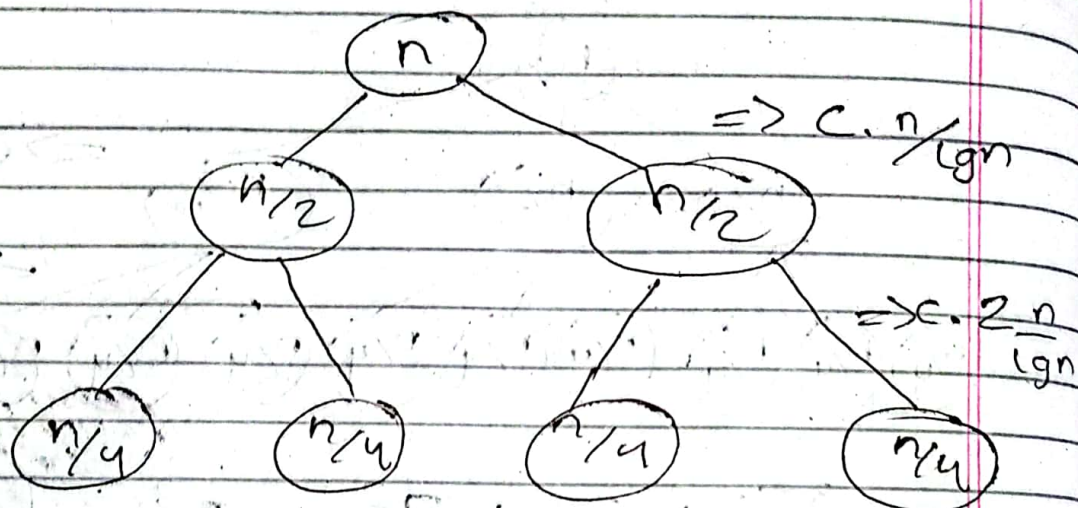
Since each level does  $\Theta(1)$  work, and there are  $\log_5(n)$  levels in the recursion tree, the total work is  $\Theta(\log_5 n)$ .

The asymptotic upper bound is  $\Theta(\log_5 n)$ .

cd

$$T(n) = 2T(n/2) + n/\lg n$$

$$\begin{aligned} \frac{n}{2} &\div \lg \frac{n}{2} \\ \frac{n}{2} &\times \frac{2}{\lg n} \\ \frac{n}{2} &\times \frac{4}{\lg n} \end{aligned}$$



$$\Rightarrow c \cdot \frac{n}{\lg n} (1 + 2 + 4 + \dots)$$

$$\Rightarrow c \cdot \frac{n}{\lg n}$$

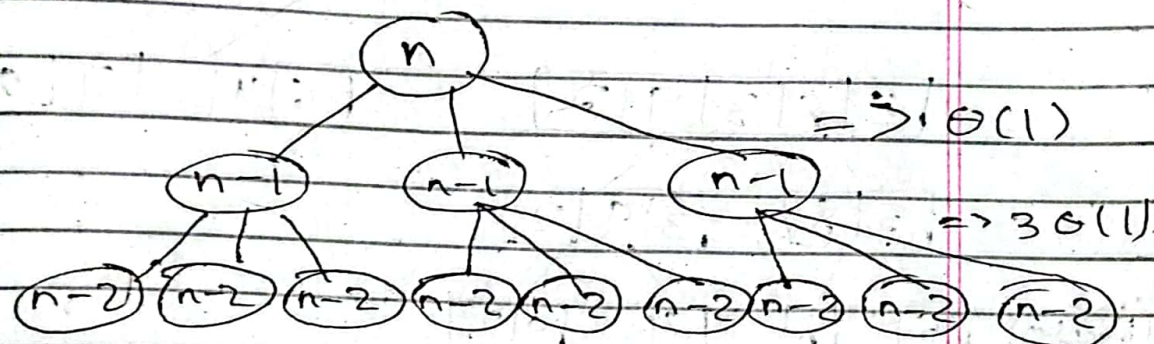
$$\Rightarrow O\left(\frac{n}{\lg n}\right)$$

The asymptotic upper bound for this recurrence tree is  $O\left(\frac{n}{\lg n}\right)$ .



Q

$$T(n) = 3T(n-1) + \Theta(1)$$



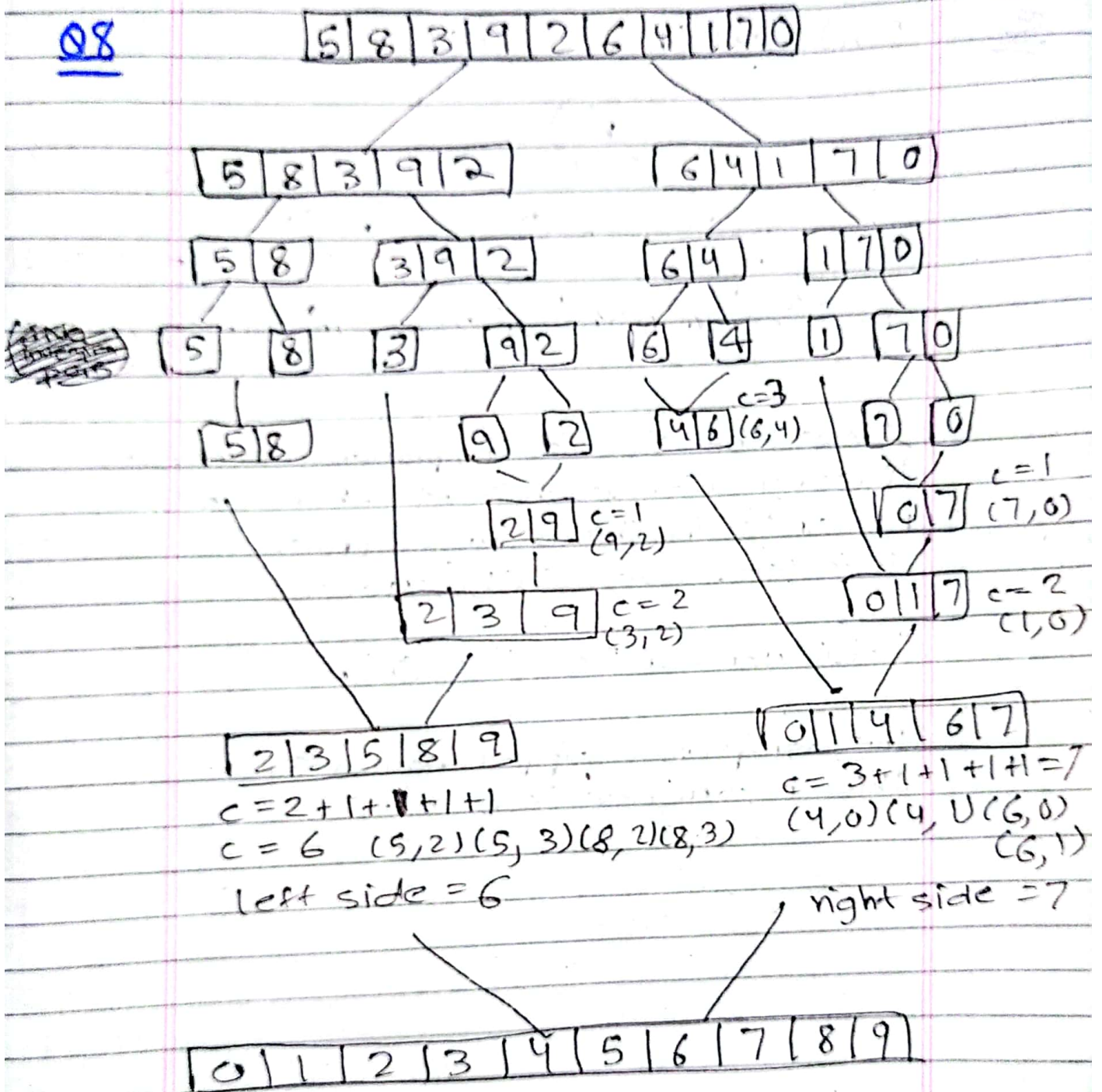
Since the size of the problem decreases linearly with each level, there are  $n$  levels in the recursion tree. Each level does  $\Theta(1)$  work.

$$\text{Total work} = n * \Theta(1) = \Theta(n)$$

The asymptotic upper bound for this recurrence is  $\Theta(n)$ .



Q8



Split pairs: -  $(2, 0)(3, 0)(5, 0)(8, 0)(9, 0) \rightarrow 5$   
 $(2, 1)(3, 1)(5, 1)(8, 1)(9, 1) \rightarrow 5$   
 $(5, 4)(8, 4)(9, 4) \rightarrow 3$   
 $(8, 6)(9, 6) \rightarrow 2$   
 $(8, 7)(9, 7) \rightarrow 2$   
 $= 17$

Total = left + right + split  
 $= 6 + 7 + 17 = 30$